

Pioneer MBR “Batch F” Single-File Hang

Root-cause walkthrough for the Sciex ZT scanning-DIA work

feat/sciex-zt-scanning

2026-07-08

1 Symptom

Every single-file ZT search stalled in the **Precursor Scoring** stage, at the match-between-runs (MBR) step, and never reached Chromatogram Integration. The process stayed alive at only ~2 cores for >14 minutes with no new log output after this line:

```
[...:20:31] [DEBUG1] MBR Batch F: writing per-file MBR sidecars...
```

Running the *same* data as a **3-file** experiment, the identical step finishes in under a second:

Files	MBR “per-file sidecar” step	Whole MBR Batch F
1 file	>14 min (never completes; killed)	—
3 files	~0.9 s (18:14:49.5 → 18:14:50.4)	~3 s

So the cost is not proportional to the data size — one file is dramatically *slower* than three. That inversion is the tell: the single-file case hits a degenerate code path, not simply “more work”.

2 Where it happens (call chain)

All paths are relative to the worktree `pio-sciex-jl (.../pio-sciex-zt/)`.

1. **Stage entry / dispatch** — `src/Routines/SearchDIA/SearchMethods/PrecursorScoringSearch/score_`
2. **MBR driver** `_score_precursor_isotope_traces_mbr`, the per-file sidecar fan-out — `.../score_psms.jl:2`
(loop at line 314)
3. **Per-file feature compute** — `.../mbr_streaming.jl:592 compute_mbr_features_per_file_to_sidecar_w`
4. **Inner per-row loop** — `.../mbr_streaming.jl:499 _compute_mbr_inner!`
5. **The hot spot: counterfactual “false donor” search** — `.../mbr_streaming.jl:416`
`_nearest_cross_file_donor` (via `_false_donor_for_pid`, line 479, and `_resolve_false_donor_for_pid`, line 443)

3 The relevant code

3.1 Dispatch: MBR on $\Rightarrow$ streaming MBR path (score_psms.jl:53)

```
53 function score_precursor_isotope_traces(  
54     second_pass_folder::String,  
55     file_paths::Vector{String},  
56     precursors::LibraryPrecursors,  
57     fragment_lookup::LibraryFragmentLookup,  
58     ::Int64, ::Float32 = 0.01f0, ::Bool = false;  
59     match_between_runs::Bool = true,  
60 )  
61     if match_between_runs  
62         return _score_precursor_isotope_traces_mbr(file_paths, precursors,  
63             fragment_lookup)  
64     else  
65         return _score_precursor_isotope_traces_no_mbr(second_pass_folder,  
66             file_paths, precursors)  
67     end  
68 end
```

With `match_between_runs=true` (the default, and what our config uses), the streaming MBR path runs *regardless of how many files there are*.

3.2 The fan-out over files (score_psms.jl:304)

```
304 @debug_l1 "MBR Batch F: building donor dict via sweep-1..."  
305 donor_dict = build_mbr_donor_dict_streaming_with_pass1(file_paths)  
306 @debug_l1 "    donor dict pids: $(length(donor_dict))"  
307  
308 @debug_l1 "MBR Batch F: writing per-file MBR sidecars..."  
309 parallel_foreach!(length(file_paths)) do chunk  
310     for f_idx in chunk  
311         compute_mbr_features_per_file_to_sidecar_with_pass1!(  
312             file_paths[f_idx], donor_dict, cf_partner_pools, fragment_keys)  
313     end  
314 end
```

Note: `parallel_foreach!` parallelises *across files*. With one file there is exactly one work item, so this is effectively single-threaded — which is why we saw only ~ 2 cores busy.

3.3 The per-row loop calls the false-donor search (mbr_streaming.jl:558)

For every PSM row, `_compute_mbr_inner!` looks up a real donor and a “false” (counterfactual) donor:

```
558 donor_f = _false_donor_for_pid(false_donor_cache, donor_dict, partner_pools,  
    my_pid, my_file)
```

3.4 The false-donor resolver (mbr_streaming.jl:479, 443)

```
479 @inline function _false_donor_for_pid(false_donor_cache, donor_dict,  
    partner_pools, my_pid, my_file)  
480     if haskey(false_donor_cache, my_pid)
```

```

481         return false_donor_cache[my_pid]           # cached per precursor id
482     end
483     donor = _resolve_false_donor_for_pid(donor_dict, partner_pools, my_pid,
484         my_file)
485     false_donor_cache[my_pid] = donor
486     return donor
end

443 @inline function _resolve_false_donor_for_pid(donor_dict, partner_pools,
444     my_pid, my_file)
445     # ... pick fold / m/z-bin for my_pid ...
446     # tries three progressively broader iRT pools, each via
447     _nearest_cross_file_donor:
448     donor = _nearest_cross_file_donor(donor_dict, <m/z-bin pool>, my_irt,
449         my_file)
450     donor != nothing && return donor
451     donor = _nearest_cross_file_donor(donor_dict, <fold pool>, my_irt,
452         my_file)
453     donor != nothing && return donor
454     return _nearest_cross_file_donor(donor_dict, <global pool>, my_irt,
455         my_file)
end

```

3.5 The hot spot (mbr_streaming.jl:416)

```

416 @inline function _nearest_cross_file_donor(donor_dict, pool, target_irt,
417     my_file)
418     n = length(pool.irts)
419     n == 0 && return nothing
420     right = searchsortedfirst(pool.irts, target_irt) # 0(log n): nearest by
421     iRT
422     left = right - 1
423     @inbounds while left >= 1 || right <= n           # <-- walks OUTWARD
424         across pool
425         use_left = ...                                # (nearest-iRT-
426             first order)
427         pool_idx = use_left ? left : right
428         use_left ? (left -= 1) : (right += 1)
429         donor = _donor_for_pid(donor_dict, pool.pids[pool_idx], my_file)
430         donor != nothing && return donor                # first CROSS-FILE hit
431         wins
432     end
433     return nothing                                     # scanned the WHOLE
434     pool, found none
end

```

3.6 The “different file” requirement (mbr_streaming.jl:406)

```

406 @inline function _donor_for_pid(donor_dict, pid, my_file)
407     entries = get(donor_dict, pid, nothing)
408     entries == nothing && return nothing
409     @inbounds for e in entries
410         e.ms_file_idx != my_file && return e           # <-- must come from
411         ANOTHER file
412     end
end

```

```

412     return nothing
413 end

```

4 Root cause

The counterfactual (“false donor”) feature is, by construction, a donor from a *different run*: `_donor_for_pid` only returns an entry whose `ms_file_idx != my_file` (line 410–411).

With a single file, no entry can ever satisfy that test. So for *every* precursor:

- `_nearest_cross_file_donor` does its `searchsortedfirst`, then walks *outward over the entire pool* (the `while` loop, line 427), calling `_donor_for_pid` on every member, and **never finds a cross-file donor**, so it scans all n pool entries and returns **nothing**.
- `_resolve_false_donor_for_pid` then repeats this for *all three* pools ($m/z\text{-bin} \rightarrow \text{fold} \rightarrow \text{global}$), the last of which contains essentially every precursor.

The `false_donor_cache` caches one result per precursor id, so this exhaustive scan happens once per *distinct* precursor. With P distinct precursors and a global pool of size $\approx P$:

$$\text{cost} \approx P \times P \times (3 \text{ pools}) = O(P^2).$$

For this data $P \approx 3.5 \times 10^5$, so $P^2 \approx 1.2 \times 10^{11}$ dictionary probes — i.e. the observed 14+ minutes. And the whole effort is wasted: the answer is always **nothing**.

With ≥ 2 files, the pool is dense with precursors that *were* seen in other runs, so the outward walk from the nearest-iRT position finds a cross-file donor within a handful of steps: $O(\log n)$ per precursor. That is why 3 files finish the same step in ~ 0.9 s.

In one sentence: MBR-across-runs is meaningless with a single file (there is nothing to match *between*), but the code still executes the full counterfactual search, which degenerates into an always-failing $O(P^2)$ scan.

5 Proposed fixes

1. **Guard at the dispatch (primary fix).** In `score_precursor_isotope_traces` (`score_psms.jl:53`), if the experiment has only one raw file, skip the MBR path and use the existing non-MBR path (`_score_precursor_isotope_traces_no_mbr`), emitting a one-line warning. This is correct: with one file the MBR features would all be “missing” anyway. (Care: files may be split into `_fold0/_fold1` paths, so count *distinct raw files*, not `length(file_paths)`.)
2. **Belt-and-suspenders (optional).** In `_nearest_cross_file_donor` (`mbr_streaming.jl:416`), short-circuit when the pool contains no file other than `my_file` (precompute the set of `ms_file_idx` present once), so any future no-cross-file-donor situation is $O(1)$ instead of $O(n)$. This leaves all multi-file behaviour unchanged.

Fix (1) resolves the reported hang directly; fix (2) hardens the kernel against the same failure mode in edge cases (e.g. a precursor whose entire iRT neighbourhood is single-file-only).